Profile-First Fitness Planning Under Real Constraints: Active Knowledge Modeling for Personalized Training Decision Support

Weishi Shao

March 2026

Abstract

Platforms such as OpenClaw, ChatGPT, and Gemini already provide persistent context surfaces for user and agent state, but they provide little guidance on how the state behind those surfaces should be elicited, structured, updated, and reused in real decision environments. OpenClaw makes this especially explicit through injected workspace files and system-prompt reconstruction. The gap becomes highly visible in fitness planning, where training decisions depend on ranked goals, injury history, recovery volatility, equipment access, and time budget. This paper presents the fitness branch of Active Knowledge Modeling (AKM) as a profile-first method for training decisions under real constraints. Instead of generating a workout plan first, the branch elicits and structures the state that should govern the decision, then routes downstream planning through an explicit execution contract. The contribution is methodological rather than outcome-validational. The paper clarifies why ordinary prompting, memory-only personalization, and split-based template logic break down in high-constraint training settings; specifies the branch workflow of elicitation, structured record, and execution decision; and shows how local design records can make workflow behavior inspectable without pretending to be broad clinical evidence.

Keywords: active knowledge modeling, fitness planning, personalized training decision support, user modeling, profile construction, context engineering, human-AI collaboration, decision support systems, training load monitoring, autoregulation

1 Introduction

Platforms such as OpenClaw, ChatGPT, and Gemini already provide persistent context surfaces for user and agent state, but they rarely provide a rigorous method for constructing the state that should populate them. OpenClaw makes this especially explicit through injected workspace files and system-prompt reconstruction. In many tasks this omission is tolerable because the downside of generic output is only mediocre usefulness. In fitness planning the omission is more consequential. A generic answer can silently overwrite constraints that should dominate the decision: injury risk, movement limits, equipment availability, low recovery, or a hard session-time ceiling.

The problem is therefore upstream. If a training decision depends on body condition, recovery state, time budget, equipment access, and adherence risk, then the system should not begin with a

plan. It should begin with a model of what can actually be advanced under the current constraints. The central AKM claim is that user understanding should become an explicit pre-task layer rather than a weak side effect of downstream dialogue.

That framing matters because fitness advice often looks more competent than it really is. A system can sound precise while quietly assuming stable readiness, generic movement capacity, or a template split that does not fit the operator. In practice, the user then pays the hidden cost through hesitation, low adherence, repeated clarification, or accumulated injury friction. Work on alignment debt helps name this hidden labor at the systems level [6], while related work on incentive structure and preference elicitation helps explain why seemingly capable agents still start too early [5, 1, 8]. The fitness branch of AKM is designed to prevent that early start.

2 Problem Landscape

2.1 Why generic fitness prompting fails

Generic fitness prompting degrades in predictable ways when upstream context is weak:

- injury or pain signals are flattened into normal programming assumptions
- equipment limitations are treated as details rather than determinants
- time budgets are treated as flexible when they are actually hard constraints
- recovery uncertainty is overwritten by split logic or template logic
- the system behaves as if confidence already exists even when key state variables are missing

These failure modes reflect a structural mismatch between what the decision actually depends on and what the workflow models explicitly. Research on training-load monitoring and autoregulation makes this point from a different angle: training quality depends on the interaction among workload, readiness, and context rather than on a fixed template alone [2, 7, 4]. Research on adherence further shows that exercise systems fail not only because the workout is suboptimal in theory, but because real-world delivery and execution conditions are mismatched to the operator [3].

2.2 Why existing platform context surfaces are still insufficient

The existence of a persistent context surface is not the same thing as a usable upstream method. In fitness, the missing pieces are usually not raw data volume but state construction discipline:

- the system does not force goal ranking before plan generation
- the system does not distinguish hard limits from soft preferences
- the system does not persist recent execution evidence as a reusable state layer

- the system does not surface uncertainty as an explicit output object

That is why fitness planning remains a strong example of the AKM thesis. Existing platform affordances allow users to store or inject context, but they do not define how the state behind those surfaces should be elicited, structured, updated, and reloaded before the next decision.

3 Branch Method

The fitness branch does not model identity for its own sake. It models training feasibility. The operator state that matters is the state that can invalidate or reshape a session.

3.1 Workflow overview

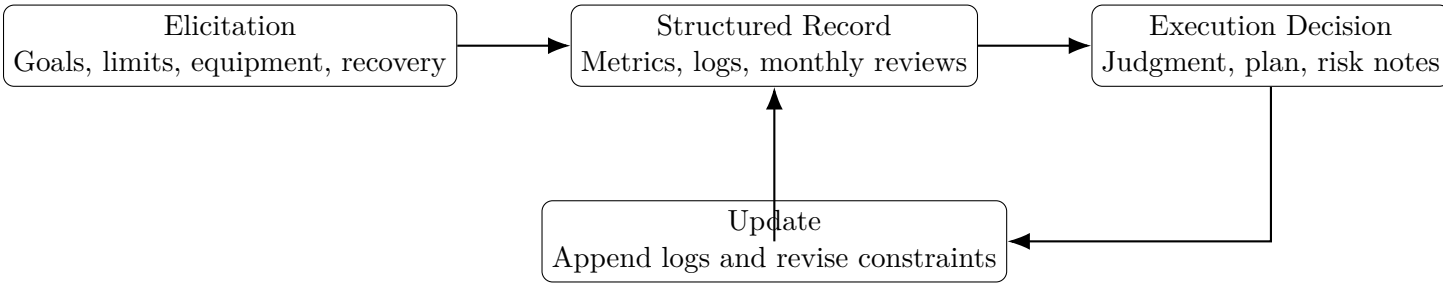

Figure 1: AKM fitness branch workflow.

The branch uses a stable four-step logic: ask what kind of session would even be legitimate, persist the answer as reusable upstream state, issue a bounded decision rather than a generic plan, and update the record after execution or non-execution.

3.2 Elicitation layer

The elicitation layer clarifies which variables are decision-critical before any recommendation is emitted. At minimum it seeks ranked goals, injury history, movement restrictions, available equipment, weekly frequency expectations, session time budget, recovery state, and adherence risk. The questions are not there to make the answer feel personalized. They are there to decide whether a recommendation should exist at all.

3.3 Structured record layer

The elicited information is converted into persistent upstream state. In this branch, state is maintained through metrics snapshots, equipment records, append-only logs, day-level decision files, and monthly retrospective summaries. That means the branch can refer to prior load, recurring constraints, or prior non-compliance without re-deriving the user from scratch each session.

Record Type	Example Source	Role in the Workflow
Rules	`rule.mdc`	defines hard boundaries, output contract, and non-negotiables
Body state	`body_metrics.md`	maintains baselines, constraints, and performance anchors
Equipment context	`equip.md`	limits exercise selection to the actual environment
Day-level framing	`today.md`	captures current state and immediate decision context
Execution log	`training_log.md`	records what actually happened, not just what was advised
Monthly review	monthly files	tracks stage-level drift, progress, and recurring bottlenecks

Table 1: Core record types in the fitness branch.

3.4 Execution decision layer

Only after profile and recent state are available does the system produce a training decision. The output contract includes `StateJudgment`, `PrimaryDecision`, `DecisionConfidence`, `Plan`, `RiskNotes`, `NonNegotiables`, and `MissingInputs`. Missing information is treated as part of the decision rather than something to hide behind fluent language.

3.5 Variable map

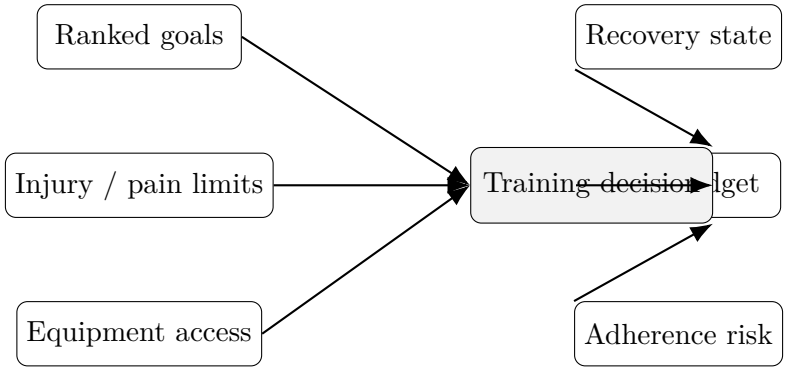

Figure 2: Decision-critical variables in the fitness branch.

Any one of these inputs can invalidate what looks like a good session on paper.

4 Design Record and Reference Trace

The local design record for this branch includes hard system rules, structured body metrics, equipment context records, day-level decision files, append-only execution logs, and monthly retrospective

records. These records are not presented as broad proof of effectiveness. They are used to make workflow behavior visible and to show that the branch is a long-running decision system rather than a single polished prompt.

4.1 Representative decision trace


```
{"StateJudgment":"Recovery state partially unclear; lower-back risk remains a live constraint.
"PrimaryDecision":"Do not assign heavy compound lifting today. Use a lower-risk session or paus
"DecisionConfidence":"Low",
"Plan":["Confirm current lumbar discomfort level and sleep quality before loading decisions.",
"RiskNotes":["Generic split logic is not sufficient under unresolved recovery and injury uncert
"NonNegotiables":["No heavy axial loading until state is clarified.","Do not infer readiness fi
"MissingInputs":["Current lumbar discomfort score (1-10)","Previous session load and residual s
```


The trace is valuable because it shows what the system does when certainty is low. It does not fake a stable program. It contracts the decision space.

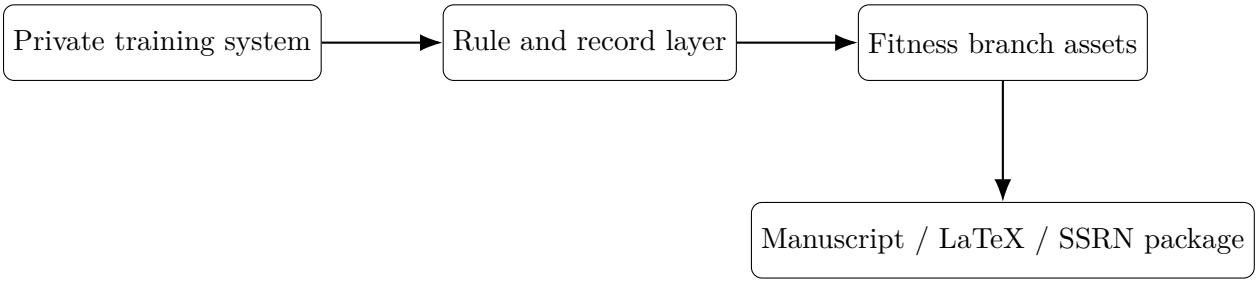

Figure 3: Evidence path from private system to public branch paper.

5 Comparative Discussion

The real comparison is not between one workout split and another. It is between different ways of handling upstream user state. This comparison is where AKM adds conceptual value. It is not merely "more personalized fitness prompting." It is a change in where the decision starts.

That shift also changes what counts as a good answer. In a generic assistant, a good answer often means a detailed session. In the AKM fitness branch, a good answer may be a refusal to issue heavy programming because the state is underdetermined. The narrower answer is the higher-quality decision.

The branch also changes how operator labor is spent. Without a profile-first layer, the user repeatedly restates goals, injury constraints, or recovery facts. With a structured upstream layer, that labor is converted into reusable context capital. This does not eliminate the need for future updates, but it changes them from repeated re-explanation to bounded state maintenance [6, 5, 1, 8].

Approach	What It Sees	Typical Failure Mode	Result Under Real Constraints
Generic prompt	current user message only	assumes normal readiness and generic equipment	fluent but fragile
Memory-only personalization	scattered prior fragments	lacks decision contract and uncertainty discipline	more context, still unstable
Template split logic	program archetype	treats schedule and body as already solved	overconfident and rigid
AKM fitness branch	explicit elicitation + persistent state + bounded output	narrower answer surface	slower but more usable

Table 2: Comparison of upstream state-handling strategies.

6 Boundaries and Failure Modes

This paper is not a medical paper. The branch does not replace physicians, rehabilitation specialists, or in-person coaching. Its claim is narrower: AKM can be implemented as a profile-first planning system in a high-constraint training environment.

The branch can still fail. The main failure modes are stale state that no longer matches current recovery or injury reality, insufficient honesty at the elicitation stage, incorrect transfer from private constraints to public branch assets, and downstream misuse when a user treats the branch as a medical authority. These are not arguments against the method. They are arguments for preserving explicit uncertainty and update discipline inside the workflow.

7 Conclusion

The fitness branch matters because it shows how AKM converts workout generation into a decision process grounded in user state, constraints, and explicit uncertainty. The contribution is methodological: better upstream modeling can matter more than better downstream phrasing.

The branch therefore supports a stronger claim than "fitness can be personalized." The stronger claim is that personalization quality depends on whether the system has first built a reusable operator model with enough structure to govern the next decision. In constrained training environments, that upstream layer is not an accessory. It is the main system.

References

[1] Marco Foschini, Marianne Defresne, Emilio Gamba, Bart Bogaerts, and Tias Guns. Preference elicitation for step-wise explanations in logic puzzles, 2025. URL `https://arxiv.org/abs/`

2511.10436.

[2] Carl Foster, Juan A. Rodriguez-Marroyo, and Jos J. de Koning. Monitoring training loads: The past, the present, and the future. *International Journal of Sports Physiology and Performance*, 12(S2):S2–2–S2–8, 2017. doi: 10.1123/ijspp.2016-0388. URL https://pubmed.ncbi.nlm.nih.gov/28253038/.

[3] A. Fuente-Vidal, M. Guerra-Balic, O. Roda-Noguera, J. Jerez-Roig, and J. Montane. Adherence to ehealth-delivered exercise in adults with no specific health conditions: A scoping review on a conceptual challenge. *International Journal of Environmental Research and Public Health*, 19 (16):10214, 2022. doi: 10.3390/ijerph191610214. URL https://pubmed.ncbi.nlm.nih.gov/36011856/.

[4] M. Haddad, G. Stylianides, L. Djaoui, A. Dellal, and K. Chamari. Session-rpe method for training load monitoring: Validity, ecological usefulness, and influencing factors. *Frontiers in Neuroscience*, 11:612, 2017. doi: 10.3389/fnins.2017.00612. URL https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2017.00612/full.

[5] Joshua Holstein, Patrick Hemmer, Gerhard Satzger, and Wei Sun. When thinking pays off: Incentive alignment for human-ai collaboration, 2025. URL https://arxiv.org/abs/2511.09612.

[6] Cumi Oyemike, Elizabeth Akpan, and Pierre Hervé-Berdys. Alignment debt: The hidden work of making ai usable, 2025. URL https://arxiv.org/abs/2511.09663.

[7] K. Shattock and J. C. Tee. Autoregulation in resistance training: A comparison of subjective versus objective methods. *Journal of Strength and Conditioning Research*, 36(3):641–648, 2022. doi: 10.1519/JSC.0000000000003530. URL https://pubmed.ncbi.nlm.nih.gov/32058357/.

[8] Jules White, Quchen Fu, Sam Hays, Peter Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt, 2023. URL https://arxiv.org/abs/2302.11382.